PROJECT SPARK — LEVEL 25 CREATION PROCESS

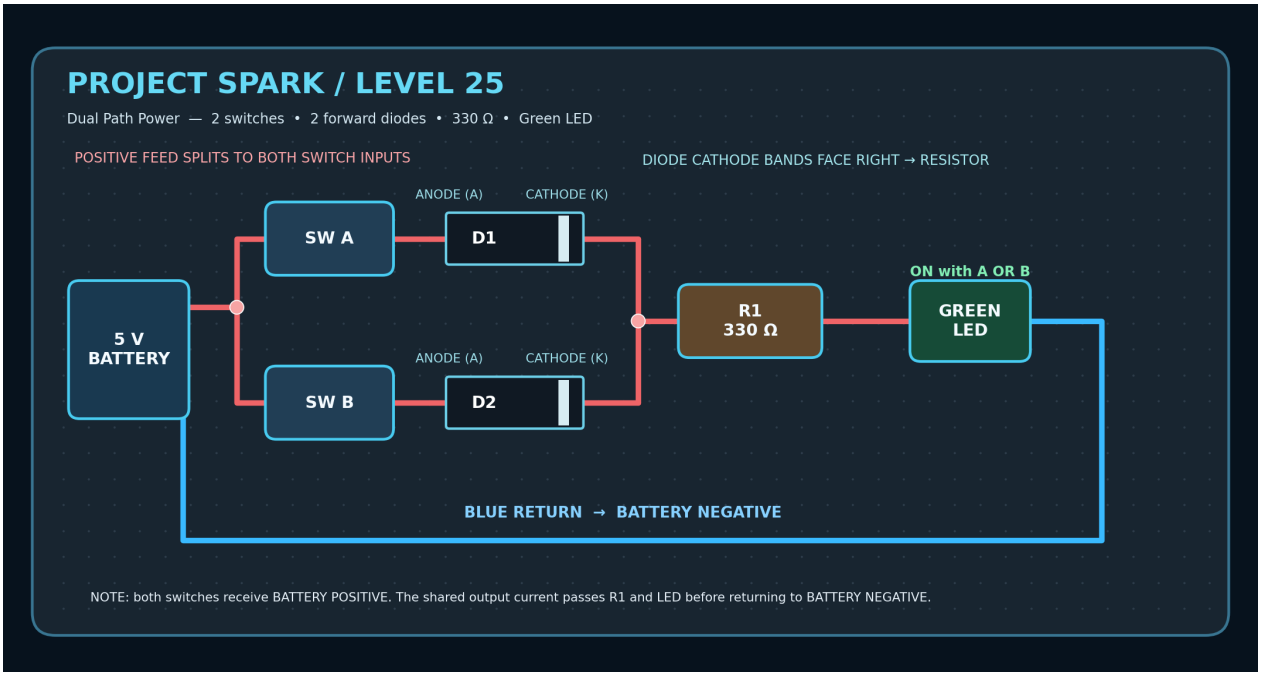
Dual Path Power — Diode OR Gate

Base implementation: 5 target evaluator / 8 external connections / 2 switches / 2 diodes / 330 Ω resistor / green LED.

Source asset: User-provided Level_25_Editable.txt, corrected while preserving its original GUID and serialized field names.

0. Before starting

Back up the working Level 24 prefab and scripts. The provided TXT updates only the original Level_25.asset using your existing Project Spark SO TXT importer. It does not create a prefab or place physical wires. Do not modify the shared electrical solver, diode solver, or Smart Slot bridge. In Level 25 the switch/diode branch wiring is different from Level 24.



1. Make the layout

Duplicate Level_24_Layout as Level_25_Layout. Use the original prefab, not the Play Mode clone. Keep the existing 5 V supply, shelf-compatible Slot_01 (resistor), Slot_02 (diode), Slot_03 (diode), and fixed green LED. Replace the old opposing-diode wiring with two forward diodes. Add a second SparkSwitch GameObject (Switch_B) with its own distinct SparkTerminal input/output. Keep the first as Switch_A. Check that both switches are discoverable by the electrical solver and respond independently; do not duplicate terminal references from A to B.

Suggested prefab hierarchy

Level_25_Layout

■■■ Power_Sources/PowerSupply_01 (5 V)

■■■ Switch_A/Power_Switch_A (SparkSwitch)

■ ■■■ SwitchA_Input

■ ■■■ SwitchA_Output

■■■ Switch_B/Power_Switch_B (SparkSwitch)

■ ■■■ SwitchB_Input

■ ■■■ SwitchB_Output

■■■ Slot_01/Resistor_330 Ω (SparkResistor)

■■■ Slot_02/Diode_1n4007_01 (SparkDiode)

■■■ Slot_03/Diode_1n4007_02 (SparkDiode)

■■■ LED/led (SparkLED)

The switch terminal names are suggestions—match actual SparkTerminal objects and verify the switch's serialized InputTerminal/OutputTerminal references. All three workstation slots should have Kind=Workstation, correct SlotId, Enable Smart Binding=ON, and occupancy registered when preinstalled.

2. Physical wiring — exactly eight connections

#	From physical terminal	To physical terminal	Logical binding name when replaceable
1	Battery_positive	SwitchA_Input	fixed → fixed
2	Battery_positive	SwitchB_Input	fixed → fixed
3	SwitchA_Output	Slot_02 Diode_Anode	D_OR_A_Anode
4	SwitchB_Output	Slot_03 Diode_Anode	D_OR_B_Anode
5	Slot_02 Diode_Cathode	Slot_01 Register_Input	D_OR_A_Cathode → R_LIMIT_Input
6	Slot_03 Diode_Cathode	Slot_01 Register_Input	D_OR_B_Cathode → R_LIMIT_Input
7	Slot_01 Register_Output	LED_Anode	R_LIMIT_Output → fixed LED
8	LED_Cathode	Battery_negative	fixed LED → fixed supply

Critical: Both switch inputs go to Battery_positive, never one to battery negative. Both diode cathode (banded) terminals merge at the input of R1. The return path is only LED_Cathode → Battery_negative. Sharing battery-positive and resistor-input terminals requires multi-wire fan-in/out support (already demonstrated in Level 24). The wiring schematic is in Visuals/Level25_Scene_Style_Wiring.png.

3. Smart Slot IDs (on ORIGINAL prefab)

Slot	Category	Runtime Component ID	Terminal A	Terminal B
Slot_01	Resistor	`R_LIMIT_Component`	`R_LIMIT_Input`	`R_LIMIT_Output`
Slot_02	Diode	`D_OR_A_Component`	`D_OR_A_Anode`	`D_OR_A_Cathode`
Slot_03	Diode	`D_OR_B_Component`	`D_OR_B_Anode`	`D_OR_B_Cathode`

SparkLevelWorkstationBindingBridge should have Use Smart Slots=ON. The legacy Workstations array may remain empty when each registered SparkShelfSlot owns its runtime IDs. Preserve the diode-aware _Anode / _Cathode rule introduced for Level 23.

Required runtime declarations in SparkLevelSceneBindings

Runtime Component IDs (3): R_LIMIT_Component, D_OR_A_Component, D_OR_B_Component.

Runtime Terminal IDs (6): R_LIMIT_Input, R_LIMIT_Output, D_OR_A_Anode, D_OR_A_Cathode, D_OR_B_Anode, D_OR_B_Cathode.

Do this in the ORIGINAL prefab outside Play Mode. Do not rely on an import action that reports “Rules checked: 0”. Manually verify the lists. Preserve fixed scene bindings for PowerSupply_01, Battery_positive, Battery_negative, led, LED_Anode, LED_Cathode and both switches. The switch terminal IDs need not be runtime IDs if switches are fixed.

4. Import corrected Level Definition

Select the original Level_25.asset in Unity. Use your PROJECT_SPARK_EDITABLE_SO_TXT_V1 importer to import Definition/Level_25_Diode_OR_Gate_FIXED.txt. The original GUID and asset path header are preserved. Confirm the asset shows level_25, 5 targets, completionMode 0, Require Closed Return=ON, Reject Short Circuit=ON and Reject Target Short=ON.

Base completion: Switch A ON / Switch B OFF

Target ID	Type	Requirement
`level25_supply`	VoltageRange	Supply 4.8–5.2 V
`level25_diode_a`	VoltageRange	D1 0.45–1.0 V
`level25_led_on`	LEDOn	Green LED illuminated
`level25_safe_current`	CurrentRange	Shared R1 current 5–8 mA
`level25_resistor_voltage`	VoltageRange	R1 drop 2.0–2.6 V

The current definition is compatible with the five-target single-state level already designed. It does not automatically verify all four OR combinations and it does not require D2 to be forward-biased while B is open.

5. Create native Visual Teaching `.asset`

Copy Editor/SparkLevel25DrawingAssetCreator.cs into a Unity Assets/Editor folder. In Unity choose Tools → Project Spark → Visual Teaching → Create Level 25 Drawing Asset. Save to Level 25's folder as Level_25_Drawing.asset. Assign it to the Visual Teaching configuration for Level 25. Four pages cover overview, eight wires, OR truth table, and five targets. The creator mirrors the supplied Level 22 asset schema and does not alter the electrical graph.

6. Truth-table functional testing

Switch A	Switch B	Expected LED	Action
OFF	OFF	OFF	Verify LED current near zero

```
| ON | OFF | ON | Verify D1 forward and 5/5 base objectives |
| OFF | ON | ON | Verify D2 forward and stable safe current |
| ON | ON | ON | Verify no source short and safe shared resistor current |
```

Test each combination by toggling switches in Play Mode and observing a new successful solve. In the both-ON case, per-branch currents may differ slightly; the common resistor limits total current. Do not assert that all four states have been passed just because the level shows 5/5. For mandatory truth-table completion, a separate level-specific state tracker must be integrated with level progression; it is not part of this base SparkLevelDefinition.

7. Final scene/solver checks

• Ensure the Level 25 layout is in SparkLevelLayoutCatalog and the LevelGamePlayManager array includes level_25; configure progression unlock from Level 24.

• Verify Active layout: True, Smart Slot rule=VALID, and all nine runtime IDs actually bind. A single undeclared ID makes the bridge fail closed.

• Verify switch A and B use distinct SparkSwitch instances and valid serialized terminals.

• Verify diode anode-to-cathode orientation for both D1 and D2, with cathode band toward R1.

• Expect roughly 5 - 0.7 - 2.0 = 2.3 V across R1, 2.3 / 330 ≈ 6.97 mA (approximate; use solved values).

• If SolveNow() fails, inspect nonlinear iteration states without permanently enabling per-device logging; preserve corrected AddForwardDrop(..., resistance).

• If 5/5 fails, inspect individual failed evaluator targets, not just the overall target count.

8. Deliverables

Definition/Level_25_Diode_OR_Gate_FIXED.txt — import-ready SO data; Editor/SparkLevel25DrawingAssetCreator.cs — Unity editor native drawing creator; Visuals/Level25_Scene_Style_Wiring.png — diagram informed by your dark bench scene; Guide/Level25_Creation_Process.md and PDF — full instructions.